

Easy Idle Game overview and architecture

Table of contents

- [How the runtime is organized](#)
- [Basic economy loop](#)
- [Required and optional managers](#)
- [Asset and assembly layout](#)
- [Important project conventions](#)
- [Choose the right guide](#)

How the runtime is organized

The core design has three layers:

1. **Definition assets** — `Business`, `Currency`, `Manager`, `Upgrade`, `Boost`, `Location`, `ShopItem`, `Achievement`, and related `ScriptableObject`s hold authored rules.
2. **Runtime holders** — types such as `BusinessHolder`, `CurrencyHolder`, and `UpgradeHolder` keep mutable state: current amount, lifetime amount, bought amount, levels, active rounds, pending queues, or unlock state.
3. **Scene managers** — components such as `BusinessesManager`, `CurrencyManager`, and `UpgradesManager` own holders, expose operations, raise events, and participate in save/load and prestige flows.

UI scripts read managers and holders through the public API. The core runtime does not depend on a particular screen layout.

Basic economy loop

The player buys businesses with currency, businesses produce rewards over time, and those rewards are reinvested into more businesses, upgrades, managers, and boosts that raise output. Production can be driven manually, as in a clicker, or run automatically once the player unlocks automation. Optional systems — locations, shops, offers, daily rewards, achievements, prestige, and offline profit — extend this loop without changing its core.

Required and optional managers

`BusinessesManager` and `CurrencyManager` are the required runtime managers. Their `Instance` accessors throw when no component exists.

Add optional managers only when the game uses their feature:

Manager	Add it when you use
<code>ManagersManager</code>	manager purchases, automation, passive or active manager modifiers
<code>UpgradesManager</code>	permanent upgrades, contextual effects, overrides, or timed queues
<code>BoostsManager</code>	inventory boosts, active boost timers, auto-production overrides, or time skip
<code>LocationsManager</code>	worlds/locations, travel, starter rewards, location locks
<code>ShopManager</code>	in-game purchases, ad/IAP handoff, consumable reward items
<code>OffersManager</code>	scheduled weighted limited-time offers
<code>PlayerStats</code>	player XP, lifetime totals, and custom stats
<code>AchievementsManager</code>	achievement locks, tiers, and claims
<code>DailyRewardsManager</code>	exact-day and repeating calendar rewards
<code>PrestigeManager</code>	reset rules, persistent prestige modifiers, and prestige rewards

SaveAndLoad	JSON persistence and load-time offline profit
ProductionModifierManager	inspector-assigned production modifier assets

The manager singleton is still a serialized scene component. Put one instance of each used manager on a persistent GameObject and avoid duplicate instances across loaded scenes.

Asset and assembly layout

Assembly	Purpose	Notable dependency
EasyIdleGame	Runtime economy and persistence	Unity runtime
EasyIdleGame.UI	uGUI, TextMeshPro, UI Toolkit presenters, demo controllers	EasyIdleGame, TextMeshPro
EasyIdleGame.Editor	Custom inspectors, menu commands, tests	Runtime and UI assemblies

Folder	Contents
EasyIdleGame > Scripts	Core runtime API
EasyIdleGame > Scripts_UI	uGUI displayers, list displayers, audio, demo managers, and UI Toolkit presenters
EasyIdleGame > Scripts_Editor	Inspectors, menu commands, Edit Mode tests, and Play Mode tests
EasyIdleGame > Demos	DocsDemo, AdventureCommunist, experimental scenes, and focused feature demos
EasyIdleGame > md > START_HERE_README.md	Open-first documentation index
EasyIdleGame > Documentation > md	Canonical prose sources

Important project conventions

Field-level help lives in the Inspector

Serialized fields are documented where you edit them: tooltips, comment areas, and conditional validation warnings on every definition asset and manager component. These guides explain workflows, relationships, and non-obvious behavior; select the asset for individual field meanings and defaults.

Put persisted definition assets below a Resources folder

Save data identifies ScriptableObject by `name` and restores them with `Resources.LoadAll<T>("")`. Every persisted definition asset must therefore be under a folder named `Resources`.

Give persisted assets unique, stable names

Renaming a definition asset changes its save identifier. Duplicate names of the same type are ambiguous and cause the loader to log an error. Treat asset names as save-schema identifiers, not just editor labels.

Prefer typed groups and Metadata

Legacy string business groups and legacy name/icon/description fields remain for migration. Create new content with typed group assets and the `Metadata` object.

Drive state through managers and holders

Use manager operations such as `TryBuyItems`, `ForceBuyItemsForFree`, `TryMergeBusinesses`, `TryTravelTo`, `CompleteExternalPurchase`, and `TryPrestigeGame`. Directly editing holder fields can bypass events, lifetime totals, bought totals, caps, sounds, and save-worthy notifications.

Choose the right guide

- Start with [Create your first idle game](#) (01-create-your-first-idle-game.pdf).
- Look up authored fields in [Core economy systems](#) (02-core-economy-systems.pdf).
- Implement optional systems with [Feature guides](#) (03-feature-guides.pdf).
- Design load and reset behavior with [Persistence, offline profit, and prestige](#) (04-persistence-offline-prestige.pdf).
- Integrate through code with [Scripting reference](#) (05-scripting-reference.pdf).
- Find a working sample in the [Demo catalog](#) (06-demo-catalog.pdf).
- Diagnose problems with [Troubleshooting](#) (07-troubleshooting.pdf).
- Build or reuse UI with [UI and displayers](#) (08-ui-and-displayers.pdf).
- Upgrade old content with the [Migration guide](#) (EasyIdleGame > md > MigrationGuide.md) and [Breaking changes](#) (BreakingChanges.pdf).